#10 Regular Expression Matching

Một dấu * có nhiều lựa chọn. Chia nhỏ thành câu hỏi **dfs(i, j)**, rồi **nhớ câu đã trả lời.**

ĐỀ BÀI TRONG 1 CÂU

Ghép `s` với `p`, phải khớp **toàn bộ** s. `.` = 1 ký tự bất kỳ; `x*` = **0 hoặc nhiều** ký tự x liền nhau.

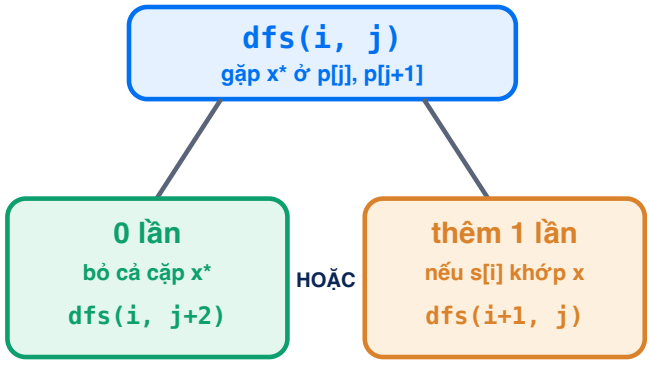
`s="aa" p="a" → false` `s="aa" p="a*" → true`

`s="ab" p=".*" → true`

CHÌA KHOÁ

Bài toán lớn = nhiều câu hỏi nhỏ `(i, j)`: "s[i:] có khớp p[j:] không?". Cùng một câu hỏi chỉ trả lời một lần (`@cache`).

HÌNH MINH HOẠ · HAI LỰA CHỌN KHI GẶP X* VÀ BẢNG NHỚ



đúng nếu một trong hai nhánh đúng

Bảng nhớ: s = "aab", p = "c*a*b"

i ↓ j →	c	*	a	*	b	ø
a	T		T		F	
a			T		F	
b			T		T	
ø						T

Chỉ 8/24 ô được tính, mỗi ô một lần

CÁCH LÀM

1. Hết p (`j == len(p)`) → chỉ đúng khi cũng hết s.
2. `first` = s[i] khớp p[j] (hoặc p[j] là `.`).
3. Nếu `p[j+1]` là `*`: thử **0 lần** (`dfs(i, j+2)`) **hoặc** dùng thêm 1 lần (`first and dfs(i+1, j)`).
4. Ngược lại: `first and dfs(i+1, j+1)`.

CODE PYTHON 3 (ĐÃ CHẠY THỬ TRÊN VÍ DỤ CỦA ĐỀ)

```python
from functools import cache

class Solution:
    def isMatch(self, s: str, p: str) -> bool:
        @cache
        def dfs(i: int, j: int) -> bool:
            if j == len(p):
                return i == len(s)
            first = i < len(s) and (s[i] == p[j] or p[j] == ".")
            if j + 1 < len(p) and p[j + 1] == "*":
                return dfs(i, j + 2) or (first and dfs(i + 1, j))
            return first and dfs(i + 1, j + 1)
        return dfs(0, 0)
```

ĐỘ PHỨC TẠP

Thời gian O(m·n)

Bộ nhớ O(m·n)

m = len(s), n = len(p): tối đa ~m·n trạng thái, mỗi trạng thái giải một lần.

BẪY HAY GẶP

- ✗ Quên x* có thể khớp **0** ký tự: `"aab"` với `"c*a*b"`.
- ✗ Đọc `p[j+1]` mà không kiểm tra `j + 1 < len(p)`.
- ✗ Không nhớ kết quả → số nhánh tăng rất nhanh.

LUYỆN TIẾP

- #44 Wildcard Matching
- #72 Edit Distance
- #97 Interleaving String

Xem video giải thích từng bước: youtu.be/jw15lSSuizM · kieu.app

Đề bài thuộc LeetCode (leetcode.com/problems/regular-expression-matching). Hình, giải thích và code do Kieu Studio biên soạn.

Nhớ cách nghĩ, đừng nhớ code.